



Laboratorio de Computación

Salas A y B



Profesor(a): Ing. Ariel Adara Mercado Martínez

Asignatura: Estructuras De Datos y Algoritmos I

Grupo: 05

No de Práctica(s): Proyecto final

Integrante(s): Tobilla González Benjamín Luis Ángel

No. de lista o brigada: 36 en lista

Semestre: 2026-2

Fecha de entrega: 11 de mayo de 2026

Observaciones:

CALIFICACIÓN: _____

Índice

Objetivo	2
Introducción	2
Desarrollo	2
Conclusiones	10

● Objetivo

Desarrollar un simulador de sistema operativo capaz de administrar: procesos, memoria, planificación de ejecución, recursos, estructuras dinámicas y algoritmos de optimización. El objetivo principal es demostrar uso correcto de estructuras de datos e implementación de algoritmos.

● Introducción

El proyecto integra los conocimientos adquiridos durante el curso de: estructuras de datos lineales, diseño de algoritmos, complejidad computacional, recurrencias, programación modular, integración entre lenguajes y desarrollo asistido por IA.

Se desarrolla un simulador de sistema operativo implementado en C con soporte de análisis y benchmarking en Python. El simulador modela los tres pilares fundamentales de un sistema operativo moderno. Además se implementan los paradigmas algorítmicos solicitados: fuerza bruta, greedy, backtracking, divide y vencerás y programación dinámica (top-down, bottom-up e incremental).

● Desarrollo

Arquitectura del sistema

Estructura de directorios:

```

1 mini-os-project/
2 +-- include/      Headers publicos (stack.h, queue.h, ...)
3 +-- src/
4 |   +-- stack.c      Pila enlazada
5 |   +-- queue.c      Cola FIFO enlazada
6 |   +-- circular_queue.c Cola circular con expansion dinamica
7 |   +-- memory/      first_fit.c - best_fit.c - worst_fit.c
8 |   |               coalescence.c - compactation.c
9 |   +-- scheduler/   fifo.c - round_robin.c - sjf.c
10 |  +-- algorithms/   brute_force.c - greedy.c - backtracking.c
11 |  |               divide_conquer.c - dp_topdown.c
12 |  |               dp_bottomup.c - incremental.c
13 |  +-- utils/        logger.c - timer.c - parser.c
14 +-- tests/          Suite de pruebas unitarias
15 +-- scripts/         Python: benchmark - graphs - validate
16 +-- reports/         CSV - PNG - PDF generados

```

Diagrama de módulos:

```

1          main.c
2          /      |      \
3 scheduler/      memory/  algorithms/
4   |             |             |

```

5	stack+queue	utils/	Python
6	circular_queue	logger+timer	benchmark+graphs

Estructuras de datos

Proceso: la unidad central del simulador, no corresponde a un proceso real del SO.

```

1 typedef enum { READY, RUNNING, BLOCKED, FINISHED } ProcessState;
2 typedef struct {
3     int pid;                // Identificador unico
4     int burst_time;         // Tiempo total de CPU requerido
5     int remaining_time;     // Tiempo restante de ejecucion
6     int priority;           // Prioridad del proceso
7     int memory_required;    // Memoria solicitada en bytes
8     ProcessState state;     // Estado actual
9 } Process;
```

Pila (Stack): implementada con lista enlazada de nodos. Uso principal: historial de ejecución y soporte a backtracking.

```

1 void stack_push(Stack* s, int value) {
2     struct Node* node = malloc(sizeof(struct Node));
3     node->value = value;
4     node->next = s->top;
5     s->top = node;
6     s->size++;
7 }
8 int stack_pop(Stack* s) {
9     struct Node* tmp = s->top;
10    int val = tmp->value;
11    s->top = tmp->next;
12    free(tmp);
13    s->size--;
14    return val;
15 }
```

Operación	Temporal	Espacial
push	$O(1)$	$O(1)$
pop	$O(1)$	$O(1)$
peek	$O(1)$	$O(1)$
destroy	$O(n)$	$O(1)$
Total	—	$O(n)$

Cuadro 1: Complejidad — Pila

Cola (Queue): cola FIFO con punteros head/tail. Uso obligatorio: scheduler FIFO.

Operación	Temporal	Espacial
enqueue	$O(1)$	$O(1)$
dequeue	$O(1)$	$O(1)$
destroy	$O(n)$	$O(1)$
Total	—	$O(n)$

Cuadro 2: Complejidad — Cola

Recurrencia del recorrido de destrucción:

$$T(n) = T(n - 1) + O(1) \Rightarrow T(n) = O(n)$$

Cola Circular (CircularQueue): implementación sobre un arreglo con índices modulares. Cuando se llena, se expande duplicando la capacidad.

```

1 // Enqueue: O(1) amortizado
2 q->data[q->tail] = value;
3 q->tail = (q->tail + 1) % q->capacity; // indice circular
4
5 // Dequeue: O(1)
6 int value = q->data[q->head];
7 q->head = (q->head + 1) % q->capacity;
```

Operación	Temporal	Espacial
enqueue	$O(1)$ amortizado	$O(1)$
dequeue	$O(1)$	$O(1)$
Expansión ($\times 2$)	$O(n)$ ocasional	$O(n)$
Total	—	$O(n)$

Cuadro 3: Complejidad — Cola Circular

Lista doblemente ligada (MemoryBlock): los bloques de memoria forman una lista doblemente ligada, lo que permite navegación bidireccional en $O(1)$ por nodo, esencial para la coalescencia.

Administración de memoria

Estrategias de asignación: el sistema parte de un único bloque libre y divide bloques cuando es necesario.

```

1 typedef struct MemoryBlock {
2     int start; // Direccion inicial
3     int size; // Tamano del bloque
4     int free; // 1 = libre, 0 = ocupado
5     int pid; // PID propietario (-1 si libre)
6     struct MemoryBlock* next;
7     struct MemoryBlock* prev; // navegacion bidireccional
8 } MemoryBlock;
```

Estrategia	Recorre	Para antes	Objetivo
First Fit	$O(n)$	Sí	Velocidad
Best Fit	$O(n)$	No	Mín. desperdicio
Worst Fit	$O(n)$	No	Máx. remanente

Cuadro 4: Comparativa de estrategias de asignación

Recurrencia (First Fit):

$$T(n) = T(n-1) + O(1) \Rightarrow T(n) = O(n)$$

Coalescencia de bloques: une bloques libres adyacentes para combatir la fragmentación externa. Se usa la lista doblemente ligada para detectar vecinos en $O(1)$ por nodo.

```

1 MemoryBlock* current = mm->head;
2 while (current && current->next) {
3     if (current->free && current->next->free) {
4         MemoryBlock* to_merge = current->next;
5         current->size += to_merge->size;
6         current->next = to_merge->next;
7         if (to_merge->next) to_merge->next->prev = current;
8         free(to_merge);
9         // No avanzar: puede haber otro bloque libre contiguo
10    } else {
11        current = current->next;
12    }
13 }
```

Ejemplo visual:

Antes: [P1:200] [Libre:300] [Libre:424] → fragmentado
Después: [P1:200] [Libre:724] → coalescido

Complejidad: $O(n)$ temporal, $O(1)$ espacial.

Compactación: reorganiza todos los bloques ocupados al inicio de la memoria, dejando un único bloque libre al final.

- Dividir: separar bloques ocupados de libres.
- Reubicar: mover bloques ocupados contiguamente desde la dirección 0.
- Combinar: crear un único bloque libre al final.

Planificación de CPU

FIFO — First In, First Out: el proceso que llega primero se ejecuta primero sin interrupciones; se usa la cola simple (Queue). Complejidad: add $O(1)$, next $O(1)$, total $O(n)$.

Round Robin: cada proceso recibe un quantum q de tiempo de CPU. Si no termina, se reinserta al final de la cola circular. Ejemplo con $q = 2$, P1=5, P2=3, P3=4:

Turno	Proceso	Ejecutado	Restante
1	P1	2	3
2	P2	2	1
3	P3	2	2
4	P1	2	1
5	P2	1	0 (termina)
6	P3	2	0 (termina)
7	P1	1	0 (termina)

SJF — Shortest Job First: elige siempre el proceso con menor `burst_time`. Es un algoritmo Greedy que minimiza el tiempo de espera promedio.

Algoritmos

Fuerza Bruta: exploración exhaustiva sin heurísticas. Garantiza encontrar el primer bloque válido pero con costo $O(n)$.

$$T(n) = T(n-1) + O(1) \Rightarrow T(n) = O(n)$$

Greedy — decisión localmente óptima: en cada paso se toma la decisión que parece mejor localmente. Aplicaciones: First Fit, Best Fit, Worst Fit (primer/menor/mayor bloque libre) y SJF (proceso de menor burst).

Backtracking: exploración con vuelta atrás: si una asignación no es válida, se deshace y se prueba la siguiente opción. Aplicación: encontrar todas las asignaciones válidas de procesos a bloques de memoria.

```

1 static int bt_assign(BTState* s, int idx) {
2     if (idx == s->n_processes) {
3         solutions_found++;
4         return 1;    // Solucion valida encontrada
5     }
6     for (int b = 0; b < s->n_blocks; b++) {
7         if (s->block_free[b] && s->blocks[b] >= s->sizes[idx]) {
8             s->block_free[b] = 0;    // asignar
9             s->assigned_block[idx] = b;
10            bt_assign(s, idx + 1);    // recursion
11            s->block_free[b] = 1;    // deshacer (backtrack)
12            s->assigned_block[idx] = -1;
13        }
14    }
15    return 0;
16 }

```

Recurrencia:

$$T(n) = n \cdot T(n-1) + O(1) \Rightarrow T(n) = n \cdot (n-1) \cdot T(n-2) + \dots = n! \cdot T(0) = O(n!)$$

En la práctica, la poda reduce drásticamente el espacio de búsqueda.

Programación dinámica — Bottom-Up (Tabulación): construye la tabla iterativamente; evita el overhead de la pila de recursión.

```

1 for (int i = 1; i <= n; i++) {
2     int req = mem_req[i - 1];
3     for (int m = 0; m <= total_mem; m++) {
4         dp[i][m] = dp[i-1][m];           // no tomar proceso i
5         if (req <= m) {
6             int with = req + dp[i-1][m - req];
7             if (with > dp[i][m]) dp[i][m] = with;
8         }
9     }
10 }

```

Algoritmo Incremental: construye la solución elemento por elemento manteniendo una invariante.

Aplicaciones:

- Insertion Sort por prioridad: $T(n) = T(n - 1) + O(n) \rightarrow O(n^2)$.
- Estadísticas en tiempo real (media de Welford): $O(1)$ por elemento.

Análisis de Complejidad — Resumen

Componente	Operación	Temporal	Espacial
Stack	push / pop / peek	$O(1)$	$O(n)$
Queue	enqueue / dequeue	$O(1)$	$O(n)$
Circular Queue	enqueue / dequeue	$O(1)^*$	$O(n)$
First Fit	allocate	$O(n)$	$O(1)$
Best Fit	allocate	$O(n)$	$O(1)$
Worst Fit	allocate	$O(n)$	$O(1)$
Coalescencia	coalesce	$O(n)$	$O(1)$
Compactación	compact	$O(n \log n)$	$O(n)$
FIFO Scheduler	add / next	$O(1)$	$O(n)$
Round Robin	tick (un proceso)	$O(1)$	$O(n)$
SJF	add / next	$O(n) / O(1)$	$O(n)$
Fuerza Bruta	búsqueda	$O(n)$	$O(1)$
Greedy (Bubble)	ordenar	$O(n^2)$	$O(1)$
Binary Search	buscar	$O(\log n)$	$O(1)$
Backtracking	exploración	$O(n!)$	$O(n)$
Merge Sort (D&V)	ordenar	$O(n \log n)$	$O(n)$
DP Top-Down	asignación óptima	$O(nM)$	$O(nM)$
DP Bottom-Up	asignación óptima	$O(nM)$	$O(nM)$
Incremental (sort)	inserción ordenada	$O(n^2)$	$O(1)$

Cuadro 5: Resumen de complejidades. *amortizado; expansión dinámica ocasional $O(n)$. M = memoria total.

Recurrencias formales

Métodos utilizados:

Sustitución — Cola y Pila

$$T(n) = T(n-1) + c \Rightarrow T(n) = nc + T(0) = O(n)$$

Árbol de recurrencia — Merge Sort

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

- Nivel 0: n trabajo
- Nivel 1: $2 \times n/2 = n$ trabajo
- Nivel k : $2^k \times n/2^k = n$ trabajo
- Profundidad del árbol: $\log_2 n$

Por lo tanto: $T(n) = n \cdot \log_2 n = O(n \log n)$.

Recurrencia no homogénea — Insertion Sort

$$T(n) = T(n-1) + n, \quad T(1) = 1$$

Solución por telescopía:

$$T(n) = n + (n-1) + \cdots + 1 = \frac{n(n+1)}{2} = O(n^2)$$

Recurrencia Backtracking

$$T(n) = n \cdot T(n-1) + O(1)$$

Expansión:

$$T(n) = n \cdot (n-1) \cdot T(n-2) + \cdots = n! \cdot T(0) = O(n!)$$

Integración C/Python

El flujo de integración sigue el modelo propuesto en el enunciado:

1. **Python genera procesos** (`generate_processes.py`): crea archivos CSV con pid, burst, memory aleatorios.
2. **C ejecuta la simulación** (`bin/main`): lee los procesos, los planifica y administra memoria.
3. **C exporta resultados** (`utils/parser.c`): escribe tiempos y estadísticas en `reports/csv/`.
4. **Python analiza y grafica** (`benchmark.py` + `graphs.py`): lee los CSV y genera gráficas en PNG/PDF.

Las gráficas generadas (`graphs.py`) incluyen:

- Tiempo de ejecución vs tamaño de entrada.
- Comparación de estrategias de memoria.

- Comparación de schedulers (tiempo de espera promedio).
- Curvas de complejidad $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$.
- Diagrama de Gantt de Round Robin.

El benchmark ejecuta **3 repeticiones** por tamaño de entrada para reducir varianza y reporta promedio, mínimo y máximo:

```

1 for size in SIZES:
2     times = []
3     for _ in range(REPS):          # 3 repeticiones para reducir varianza
4         start = time.perf_counter()
5         subprocess.run([BINARY], capture_output=True)
6         elapsed_ms = (time.perf_counter() - start) * 1000
7         times.append(elapsed_ms)
8     results.append({
9         "size":      size,
10        "avg_ms":    sum(times) / len(times),
11        "min_ms":    min(times),
12        "max_ms":    max(times),
13    })

```

Prueba y validación

La suite de tests/ cubre los seis componentes principales:

Archivo	Función probada	Resultado
test_stack.c	push, pop, is_empty	PASS
test_queue.c	enqueue, dequeue (FIFO)	PASS
test_circular_queue.c	comportamiento RR	PASS
test_memory_manager.c	first_fit, free, coalescencia	PASS
test_scheduler.c	FIFO scheduler	PASS

Cuadro 6: Resultados de pruebas unitarias

Todos los tests usan `assert()` estrictos. La compilación con `-Wall -Wextra` no produce ningún warning en el código de producción.

El test de memoria verifica la reutilización de bloques liberados:

```

1 MemoryManager* mm = mm_create(1024);
2 int block1 = mm_allocate_first_fit(mm, 100);
3 int block2 = mm_allocate_first_fit(mm, 200);
4 assert(block1 >= 0);
5 assert(block2 >= 0);
6 mm_free(mm, block1);
7 int block3 = mm_allocate_first_fit(mm, 50);
8 assert(block3 >= 0);    // reutiliza el hueco de block1
9 mm_destroy(mm);

```

Compilación y ejecución

```
1 # Compilar el proyecto completo
2 make
3
4 # Ejecutar el simulador
5 ./bin/main
6
7 # Generar benchmarking (requiere Python 3 + pandas + matplotlib)
8 python3 scripts/benchmark.py
9
10 # Generar todas las graficas
11 python3 scripts/graphs.py
12
13 # Validar resultados
14 python3 scripts/validate_results.py
```

● Conclusiones

El proyecto demuestra la integración de estructuras de datos y algoritmos en un contexto aplicado al dominio de sistemas operativos:

1. **Estructuras de datos:** pila, cola, cola circular y lista doblemente ligada fueron implementadas desde cero en C, con complejidad $O(1)$ para las operaciones fundamentales.
2. **Algoritmos:** se implementaron siete paradigmas (fuerza bruta, greedy, backtracking, divide y vencerás, DP top-down, bottom-up e incremental), cada uno aplicado a un problema concreto del simulador.
3. **Complejidad:** cada componente incluye análisis formal con recurrencias resueltas por sustitución, árbol de recurrencia, telescopía y Teorema Maestro.
4. **Integración C/Python:** Python genera datos de prueba y analiza resultados; C ejecuta la simulación. El flujo es completamente automatizable con `make benchmark`.
5. **Calidad técnica:** compilación limpia con `-Wall -Wextra`, suite de tests con `assert()`, y validación automatizada en Python.

Reflexión personal — Tobilla Benjamín: Este proyecto fue muy enriquecedor porque integró prácticamente todo lo visto durante el semestre en un sistema funcional. Fue la primera vez que implementé un scheduler real en C y vi cómo la elección de la estructura de datos correcta (cola circular para Round Robin vs. cola simple para FIFO) impacta directamente en la complejidad del algoritmo. El uso de IA como asistente en la depuración y generación del Makefile aceleró el desarrollo, pero siempre fue necesario validar su output con el conocimiento del curso. Considero que este es un modelo de desarrollo profesional real.